

课程笔记

Claude Co-Work & Claude Code 深度解析

五道口纳什 & Codex

2026 年 3 月 28 日



视频作者/频道: Greg Isenberg (YouTube)

发布日期: 2026 年 1 月

视频时长: 约 42 分钟

视频链接: 未填写

目录

1 引言：Agent 时代来临	2
1.1 本章小结	2
2 Claude Co-Work 产品详解	2
2.1 产品架构与入口	2
2.2 文件操作：从整理到自动化	2
2.3 浏览器操控：从本地到云端	3
2.4 多任务并行	3
2.5 Slack 集成与团队协作	3
2.6 本章小结	4
3 安全与信任机制	4
3.1 多层安全架构	4
3.2 权限模型	4
3.3 本章小结	4
4 Skills：可复用的能力模块	5
4.1 什么是 Skill?	5
4.2 使用建议	5
4.3 本章小结	5
5 Claude Code 最佳实践	5
5.1 并行多 Claude workflow	5
5.2 始终使用最强模型	6
5.3 CLAUDE.md 是团队知识库	6
5.4 Plan Mode：先规划再执行	6
5.5 GitHub Action 与复合工程	6
5.6 给 Claude 验证自身输出的能力	7
5.7 本章小结	7
6 未来展望	7
6.1 代码的消亡与重生	7
6.2 Co-Work 的平台潜力	8
6.3 本章小结	8
7 总结与延伸	8
7.1 拓展阅读	8

1 引言：Agent 时代来临

本期节目由 Greg Isenberg 主持，邀请到 Anthropic 的 Boris——Claude Code 的创造者、Claude Co-Work 的核心推动者——做客访谈。Boris 不仅展示了 Co-Work 的操作流程，还分享了他个人使用 Claude Code 的工作流和最佳实践。本讲义将从产品定位、使用方法、安全机制和高效工作流四个维度对谈话内容进行系统梳理。

核心观点

Claude Co-Work 的本质是将 Claude Code 的 Agent 能力包装到一个人人可用的 UI 中，其底层运行的是与 Claude Code 完全相同的 Claude Agent SDK。Co-Work 可以操作文件、控制浏览器、通过 MCP 连接任意工具——它不仅仅是聊天，而是真正意义上的 Agent。

什么是 Agentic AI？

Boris 指出，“agentic”这个词在业界已被严重滥用。它的核心含义是：**AI 可以采取行动 (take action)**——不仅仅是生成文本或搜索网页，而是能够使用计算机上的工具、与真实世界交互。Anthropic 从很早期就将 coding、tool use、computer use 作为模型训练的三大核心能力方向。

1.1 本章小结

Co-Work 是 Anthropic 面向非技术用户推出的 Agent 产品，它将 Claude Code 的全部 Agent 能力通过简洁的桌面端 UI 呈现出来。理解“Agent = 能采取行动”这一核心概念，是理解 Co-Work 所有功能的基础。

2 Claude Co-Work 产品详解

2.1 产品架构与入口

Claude 桌面应用目前包含三个标签页：

1. **Chat**：默认的对话界面
2. **Co-Work**：全新的 Agent 工作区
3. **Code**：即 Claude Code 终端界面

Boris 特别强调，Co-Work 底层直接调用 Claude Agent SDK——与 Claude Code 使用的是完全相同的 Agent 引擎。目前 Co-Work 仅支持 macOS，Windows 版本即将推出。

2.2 文件操作：从整理到自动化

Boris 演示的第一个场景是**收据管理**：

1. 将桌面上的收据文件夹挂载到 Co-Work（需要用户主动授权，默认无权限）
2. 输入指令：“我有一个收据文件夹，能否将文件按日期重命名？”

3. Co-Work 读取了四张收据，发现其中一张缺少日期，主动向用户求证——Boris 将此称为 **Reverse Solicitation**（反向请求）
4. 完成重命名后，进一步将数据整理到电子表格

Reverse Solicitation（反向请求）

当模型对某些信息不确定时，它会**主动向用户提问**而非盲目假设。这一行为经过专门训练，是 Agent 安全运行的关键机制之一。对用户而言，与 Co-Work 的交互更像是在与一个认真负责的队友协作——它会在需要时寻求澄清。

2.3 浏览器操控：从本地到云端

当 Boris 提出“也许我不想要本地电子表格，我想要一个 Google Sheet”时，Co-Work 自动打开浏览器，导航到 Google Sheets，创建新表格并逐行填入数据。

浏览器能力的技术基础

Co-Work 内置了对 Chrome 浏览器的原生支持。模型可以：

- 读取屏幕内容（类似 computer use）
- 点击按钮、填写表单
- 在多个标签页之间导航

用户需要安装 Chrome Extension 并在每个站点首次访问时授权（可选择“允许一次”或“始终允许”）。

Boris 还演示了让 Co-Work 打开 Gmail 并将表格发送给同事 Amy 的流程——模型自动查找联系人、撰写邮件正文并保存草稿等待确认。

2.4 多任务并行

并行工作流是核心范式

Co-Work 和 Claude Code 最强大的使用模式不是一次完成一个任务，而是**同时运行多个任务**。当一个任务正在执行时（如操控浏览器制作表格），用户可以新建另一个任务（如搜索播客创业灵感）。Boris 日常同时运行 5–10 个 Claude 实例。

Greg 在访谈中提出了一个关键思路：应该对公司的**所有日常任务进行审计**——梳理团队如何处理文件、使用互联网、发送信息，然后识别哪些环节可以交给 Co-Work 自动化。

2.5 Slack 集成与团队协作

Boris 分享了一个他每周使用多次的实际场景：

1. 团队有一个项目追踪表格
2. Co-Work 检查表格中未填写的列

3. 自动在 Slack 上给对应工程师发消息提醒

这个流程完全由 Co-Work 自主完成——Boris 只需发出指令，然后去喝杯咖啡。

2.6 本章小结

Co-Work 的核心使用场景涵盖文件整理、浏览器操作、邮件/Slack 通信和多任务并行。对于非技术用户而言，Co-Work 去除了终端的技术门槛，同时保留了 Claude Code 的全部 Agent 能力。其安全机制（授权、reverse solicitation、沙箱）确保了操作的可控性。

3 安全与信任机制

3.1 多层安全架构

Boris 用了大量篇幅强调 Anthropic 在安全方面的投入，主要分为以下几层：

1. **模型层——对齐与可解释性**：Anthropic 的核心使命是 AI 安全。在模型训练阶段，通过 alignment（对齐）和 mechanistic interpretability（机制性可解释性）确保模型行为符合用户意图。Boris 类比说，这就像用科学方法研究人脑神经元一样研究 AI 的内部结构。
2. **运行时——虚拟机沙箱**：Co-Work 底层有一个完整的虚拟机运行环境，确保任何操作都不会影响用户的宿主系统。
3. **用户层——删除保护**：新增的 deletion protection 机制会在删除文件前提示用户确认。
4. **网络层——Prompt Injection 防护**：当模型与互联网交互时，内置了针对 prompt injection 攻击的防护措施。

安全仍在演进中

Boris 坦承这些防护并不完美，Anthropic 仍在持续迭代。尽早发布产品的重要原因之一，就是要在真实使用场景中观察和研究模型的行为模式，以进一步改进安全性。

3.2 权限模型

Co-Work 采用**最小权限原则**：

- 文件系统：默认无权访问任何文件夹，需要用户逐一挂载
- 浏览器：每个网站首次访问需要单独授权
- 操作确认：敏感操作（如发送邮件、删除文件）会在执行前求证

3.3 本章小结

Anthropic 构建了从模型内部（对齐、可解释性）到运行环境（沙箱隔离）再到用户交互（权限、确认）的多层安全体系。Co-Work 的设计哲学是“安全优先、逐步放权”。

4 Skills：可复用的能力模块

4.1 什么是 Skill？

Skill 本质上是一种**可复用的任务执行模式**。Boris 解释说，Co-Work 在之前的演示中制作 Excel 表格时，实际上加载了一个预装的 Excel skill——这就是模型知道如何正确操作 Excel 的原因。

Skill 的价值主张

如果你使用的工具或文件格式比较特殊（如 AutoCAD、Salesforce），你可以编写自定义 Skill 来教会 Claude 如何操作它。拥有更多 Skill 的 Co-Work 实例能够处理更广泛的任务，输出质量也更高。

4.2 使用建议

Boris 的建议是**先简单后复杂**：

1. 安装 Co-Work + Chrome Extension 即可开始使用
2. 不要一上来就花时间定制 Skill
3. 当发现 Co-Work 在某个具体场景下表现不佳时，再考虑编写对应的 Skill
4. Skill 更适合进阶用户作为性能优化手段

4.3 本章小结

Skill 是 Co-Work 和 Claude Code 的可扩展能力层。它让用户可以将领域知识编码为可复用的模块，从而提升 Agent 在特定任务上的表现。

5 Claude Code 最佳实践

本节内容来自 Boris 在 X/Twitter 上发布的一条病毒式传播帖文（99,000 书签），以及他在节目中的深度展开。

5.1 并行多 Claude 工作流

Boris 的工作方式

Boris 同时运行 5–10 个 Claude 实例：

- 本地终端中通常保持多个 tab，每个 tab 一个独立的 git checkout
- 当本地 tab 用完后，溢出到 Web 端继续开新任务
- 每天早上起床后通过 iOS app 启动 2–3 个 Claude 任务
- 大约一半的代码现在是通过手机端 Claude 完成的

Boris 坦言他一年前绝不会预料到自己的编码方式会变成这样——通过手机和 Web 分发任务，像管理一个 AI 团队一样在各个实例之间巡回检查。

5.2 始终使用最强模型

Boris 推荐始终使用 **Opus 4.5 with Thinking** (非 Sonnet):

为什么更大的模型反而更便宜？

这是一个反直觉的发现：虽然 Opus 4.5 的单 token 价格高于 Sonnet，但因为它更聪明，所以：

- 完成同一任务消耗的总 token 数更少
- 需要的人工修正次数更少
- 总体成本和时间往往**低于**使用较小模型

类比：雇一个高级工程师的时薪虽高，但因为不需要反复返工，总成本可能低于雇一个初级工程师。

5.3 CLAUDE.md 是团队知识库

CLAUDE.md 的最佳实践

- 团队共享一个 CLAUDE.md 文件，提交到 Git 仓库
- **全团队每周多次更新**：每当发现 Claude 犯错，立即将修正规则加入 CLAUDE.md
- 格式完全自由——就是一个普通文本文件，没有特殊语法要求
- 每个团队维护自己的 CLAUDE.md，各团队自行负责保持其更新

Boris 展示了 Claude Code 仓库的 CLAUDE.md 截图——内容非常简洁朴素，证明了“不需要复杂格式，只需要持续维护”的理念。

5.4 Plan Mode：先规划再执行

“Once the plan is good

Boris 的大多数会话以 Plan Mode 开始：

1. 进入 Plan Mode，与 Claude 反复讨论方案
2. 方案确认后，切换到 Auto-accept Edits 模式
3. Claude 通常可以一次性 (one-shot) 完成整个 PR

这与 Spec-driven Development 思路一致，但更灵活——“spec”只是一个文本形式的计划，不需要特定格式。

5.5 GitHub Action 与复合工程

Boris 的第五条建议涉及**在代码审查中持续积累团队知识**：

1. 通过 `/install-github-action` 在仓库中安装 Claude GitHub App

2. 在 PR review 中 @claude 让它执行修改或更新 CLAUDE.md
3. 这确保同一类问题**永远不需要指出两次**

复合工程 (Compound Engineering)

Boris 引用了 Dan Schipper 的“复合工程”概念：在 Meta 工作时，Boris 会维护一个电子表格记录代码审查中出现的问题，当某个问题出现 5-10 次后就编写 lint 规则来自动化检查。现在 CLAUDE.md + GitHub Action 就是这个理念的 AI 时代版本——每次发现问题，立即让 Claude 更新知识库，确保下次不再犯同样的错。

5.6 给 Claude 验证自身输出的能力

Boris 认为这是提升 Claude Code 效果的**三大关键之一**（与使用 Opus 和维护 CLAUDE.md 并列）：

“蒙眼画家”类比

如果一个画家必须戴着眼罩作画，即便技艺再高超也无法画出好作品。同理，如果 Claude 无法看到自己写的代码的运行结果，它的输出质量就会大打折扣。因此：

- 构建 Web App 时，务必启用 Chrome Extension 让 Claude 能在浏览器中查看效果
- 始终让 Claude 运行测试、启动服务器、查看模拟器输出
- 让 Claude 能够自我验证和修正，是从“尚可”到“优秀”的关键跨越

5.7 本章小结

Boris 的 Claude Code 最佳实践可以归纳为五个核心原则：

1. **并行**：同时运行多个 Claude，像管理团队一样分配任务
2. **最强模型**：始终使用 Opus 4.5 with Thinking
3. **知识积累**：持续维护 CLAUDE.md 作为团队知识库
4. **先规划**：用 Plan Mode 确保方案正确，再进入自动执行
5. **自我验证**：让 Claude 能看到并验证自己的输出

6 未来展望

6.1 代码的消亡与重生

Boris 和 Dario Amodei 一年前都预言“到年底人们将不再亲手写代码”。到 2026 年 1 月录制节目时，Boris 表示在过去两个月中，他的所有代码（每月 200-300 个 PR）100% 由 Claude Code 编写，**他没有手写过一行代码。**

指数思维 vs 线性思维

Boris 强调，人类大脑天生按线性方式思考，但 AI 能力的增长是指数级的。一年前如果按线性推算，绝不可能想到模型已经能完全替代人类编码。唯一能做出正确预测的方法是“相信指数曲线，然后沿着它画下去”。

6.2 Co-Work 的平台潜力

Greg 和 Boris 都将当前的 Agent 生态比作 iPhone App Store 的早期：

- 2008 年 App Store 刚推出时，最火的是啤酒模拟器等玩具应用
- 没人预见到 Uber、DoorDash、TikTok 会从中诞生
- Co-Work 正处于类似阶段——使用场景的爆发将超出所有人的想象

Boris 预见的趋势是：越来越多的日常琐碎工作（在 App A 和 App B 之间搬运数据、格式转换、状态汇报等）将被 Agent 接管，人类将聚焦于自己真正享受的创造性工作。

6.3 本章小结

AI Agent 的发展遵循指数曲线。Co-Work 作为 Agent 平台的意义类似于 App Store 之于移动互联网——真正的杀手级应用尚未出现，但基础设施已经就绪。

7 总结与延伸

本次访谈的核心信息可以提炼为以下几点：

1. **Co-Work 的定位**：Claude Code 的能力 + 人人可用的 UI，是非技术用户进入 Agent 世界的入口
2. **安全设计**：多层防护体系（模型对齐 → 沙箱隔离 → 权限控制 → 用户确认）
3. **工作范式转变**：从“深入单任务”转向“并行管理多个 AI 实例”，工程师的角色更像是“AI 团队的管理者”
4. **知识复合**：CLAUDE.md 是团队级的知识积累机制，确保错误不会重复出现
5. **验证闭环**：让 Agent 能够看到并验证自身输出，是提升质量的关键

7.1 拓展阅读

- Boris 的 X/Twitter 帖文：Claude Code 使用技巧（99K 书签）
- Dan Schipper：Compound Engineering 概念
- Anthropic Blog：Claude Agent SDK 文档
- Claude Code GitHub Action：/install-github-action 命令